

# **Models — deep dive**

# The lifecycle

A model isn't a file you download once — it moves through a loop:

```
CHOOSE → REGISTER → SERVE → FINE-TUNE → EVALUATE → (loop back to serve)
                        (when needed)
MLOps wraps it all: track · version · monitor
```

Two rules of thumb:

- Need new **knowledge**? → **RAG** (layer 3), not the model.
- Need new **behaviour**? → **fine-tune**. Train from scratch almost never.

## Choosing — three dimensions

You pick the best model for a **task**, a **budget**, and a **GPU** — never "the best."

- **Open vs proprietary** — runs on your hardware vs vendor API; control & privacy vs frontier ceiling.
- **Size (LLM vs SLM)** — bigger reasons more generally; smaller is cheaper, faster, and often specialized.
- **Specialization** — reasoning · tool-calling · code · language · multimodal.

A fine-tuned 3B can beat a generic 70B on its one task — the case for SLMs.

## The one number: VRAM

$\text{VRAM}(\text{weights}) \approx \text{params} \times \text{bytes}$     ( $\text{fp16} = 2 \cdot \text{fp8} = 1 \cdot \text{int4} \approx 0.5$ )

On our lab GPU — a free Kaggle **T4 (16 GB)**:

| size | fp16   | int4 (Q4) | fp16 fits? | Q4 fits? |
|------|--------|-----------|------------|----------|
| 3B   | ~6 GB  | ~2 GB     | yes        | yes      |
| 7–8B | ~15 GB | ~5 GB     | tight      | yes      |
| 14B  | ~28 GB | ~8 GB     | no         | yes      |

On 16 GB, quantization is what lets a 7B+ fit with room for the KV cache.

## Registry — what to track

Keep an honest catalog of the models in use. One table is enough; larger teams use MLflow Registry, the HF Hub, or a database.

| Field                        | Why it matters                                |
|------------------------------|-----------------------------------------------|
| <b>id / version</b>          | reproducibility — <code>model @ commit</code> |
| <b>size &amp; precision</b>  | drives the VRAM math                          |
| <b>licence</b>               | commercial use? (next slide)                  |
| <b>context length</b>        | how much it reads at once                     |
| <b>capabilities</b>          | tools? vision? languages?                     |
| <b>source &amp; checksum</b> | provenance + integrity                        |

# Licences — "open" is a spectrum

**Open-weight ≠ open-source:** you get the weights, rarely the training data/code.

| Tier        | Licence                                          | You may...                                                      | Examples                               |
|-------------|--------------------------------------------------|-----------------------------------------------------------------|----------------------------------------|
| Permissive  | MIT · Apache-2.0                                 | commercial use, fine-tune, redistribute, self-host — no strings | Qwen (Apache), DeepSeek (MIT), Mistral |
| Conditional | "community" · "modified MIT" ·<br>license: other | mostly — with limits (user caps, branding, approval)            | Llama (community, 700M-MAU clause)     |
| Restricted  | custom / non-commercial                          | research & eval only                                            | research-only releases                 |

**Economics:** weights are free; vendors monetize API / enterprise / cloud — *commoditize the complement*. A model can start permissive and **tighten later** — read the licence every release.

## Serving — engines

|          | Ollama / llama.cpp   | vLLM                      |
|----------|----------------------|---------------------------|
| Best for | one user, demos      | many users, throughput    |
| Trick    | GGUF + Q4 built in   | PagedAttention + batching |
| Feel     | snappy single answer | wins at concurrency       |

**Latency** = one fast answer · **throughput** = total tokens/s for everyone.

Two memory costs: fixed **weights** + a **KV cache** that grows with context × concurrency.

## Serving — quantization

Fewer bits per weight: a little quality lost, a lot of memory saved.

| Format     | Bits | Where                            |
|------------|------|----------------------------------|
| fp16/bf16  | 16   | training & high-fidelity serving |
| fp8        | 8    | Blackwell/Hopper serving         |
| GGUF Qk_*  | ~4–5 | Ollama / llama.cpp               |
| AWQ / GPTQ | 4    | vLLM / GPU serving               |

**Lab 1:** Qwen2.5-3B at fp16 vs Q4 on Ollama — Q4  $\approx \frac{1}{2}$  **VRAM**. On the T4 it's *not* faster (no int4 accel); on Blackwell it would be.



## Fine-tuning — teach behaviour, cheaply

Full fine-tune of 7B = weights + grads + optimizer  $\approx$  tens of GB  $\rightarrow$  not on 16 GB.

- **LoRA** — freeze the base, train tiny adapters (<1% of params)
- **QLoRA** — load the base in **4-bit**, train the adapter on top

| method | base      | VRAM (7B) |
|--------|-----------|-----------|
| LoRA   | fp16/bf16 | ~18–24 GB |
| QLoRA  | 4-bit     | ~8–12 GB  |

You learn a few million adapter numbers, not the base weights — that's why a free 16 GB GPU can do it.

**Lab 2:** turn Qwen2.5-3B into a strict-JSON order parser; track in **MLflow**.

## Evaluation — numbers, not vibes

A demo that "felt good" is not evidence. Measure before and after — and compare the base model against the tuned one.

| kind              | what                                   |
|-------------------|----------------------------------------|
| Benchmark         | MMLU / GSM8K / ARC — general ability   |
| Task eval         | your held-out set + a metric you chose |
| LLM-judge / human | preference, helpfulness                |

Watch for **contamination** (memorized benchmarks), **quant quality** (measure it), and reporting a single number without speed and cost.

**Lab 3:** base vs fine-tuned on JSON-valid % / exact-match / field accuracy.

## Training — rarely, and why

Pre-training from scratch = **trillions of tokens + a cluster** (that's why Layer 1 exists). Millions of dollars, months.

Justified only when:

- no existing model and no fine-tune can reach the goal
- a genuinely novel domain, modality, or language
- **continued pre-training** — keep training an open base on your corpus (the cheap middle ground)

Decision path, almost always: **use** → **fine-tune** → **(only then) train**.

## MLOps — the glue

Wraps the whole loop: give every step a **versioned input and output** (reproducible), and **gate** the bad outputs before they ship.

| Step             | In → Out                           | MLOps                                 |
|------------------|------------------------------------|---------------------------------------|
| <b>Register</b>  | need → model id + licence          | catalog & version                     |
| <b>Fine-tune</b> | base + dataset → adapter + metrics | track run; version adapter + data     |
| <b>Evaluate</b>  | model + held-out set → scores      | <b>gate: regression blocks deploy</b> |
| <b>Serve</b>     | approved model → endpoint          | monitor latency · quality · drift     |

The eval gate = "checkov blocks `terraform apply`", for models.

## Aside · superintelligence

The other end of the axis: one system above humans in *every* domain.

*(Nick Bostrom, Superintelligence, 2014)*

- **Orthogonality** — intelligence and goals are independent. Smarter  $\neq$  nicer.
- **Instrumental convergence** — almost any goal implies self-preservation + resource acquisition. The **paperclip maximizer**: competence aimed at the wrong objective.

Today's models are narrow — treat this as an ethical **horizon**, not a forecast.

It's why alignment + human-in-the-loop matter in miniature now.

## If we built one — four forms of agency

*Bostrom Ch. 10 — least → most autonomy. Maps onto how we ship LLMs today.*

| Form                                | Core risk                         | ≈ today              |
|-------------------------------------|-----------------------------------|----------------------|
| <b>Oracle</b> — answers only        | the answer itself manipulates     | Q&A chatbot          |
| <b>Genie</b> — one task, then stops | literal ≠ intended                | task agent           |
| <b>Sovereign</b> — open-ended, 24/7 | no off-switch if misaligned       | autonomous agent     |
| <b>Tool</b> — no goals, you drive   | optimization can still go agentic | function / tool call |

Heuristic for **L4 orchestration**: grant the least autonomy that does the job, and keep a human in the approval loop.

</content>